

如何使用 Cloud Run Jobs 和 Video Intelligence API 處理影片

來源：<https://codelabs.developers.google.com/codelabs/how-to-use-cloud-run-jobs-video-intelligence-api-process-video?hl=zh-tw>

步驟數：9

瞭解如何使用 Vertex AI 和 Video Intelligence API 建立 Cloud Run 工作，藉此說明影片中每個場景的圖片。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 啟用 API 並設定環境變數](#)
4. [4. 建立服務帳戶](#)
5. [5. 建立 Cloud Storage 值區](#)
6. [6. 建立 Cloud Run 工作](#)
7. [7. 部署及執行 Cloud Run 工作](#)
8. [8. 恭喜！](#)
9. [9. 清除所用資源](#)

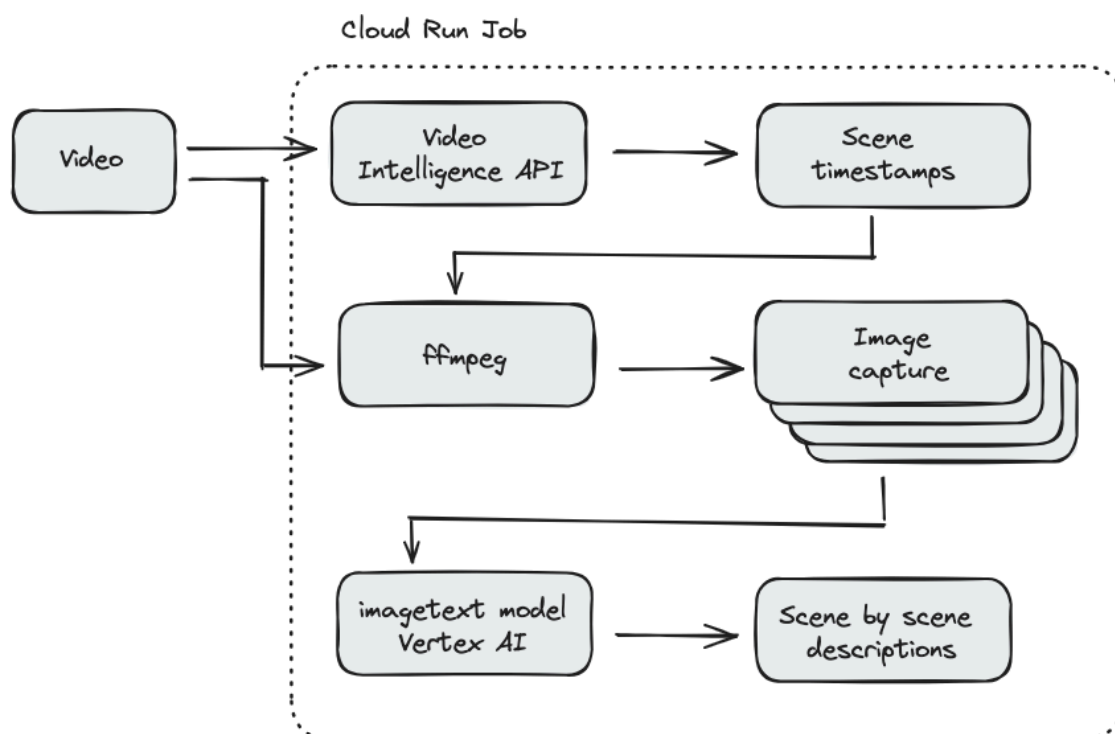
1. 簡介

總覽

在本程式碼研究室中，您將建立以 Node.js 編寫的 Cloud Run 工作，並提供影片中各場景的影像說明。首先，這項作業會使用 Video Intelligence API 偵測場景變更的時間戳記。接著，這項工作會使用名為 ffmpeg 的第三方二進位檔，擷取每個場景變更時間戳記的螢幕截圖。最後，我們使用 Vertex AI 圖像說明生成功能，提供螢幕截圖的視覺描述。

本程式碼研究室也示範如何在 Cloud Run 作業中使用 ffmpeg，從特定時間戳記的影片擷取圖片。由於 ffmpeg 必須獨立安裝，本程式碼實驗室會說明如何建立 Dockerfile，以便在 Cloud Run 作業中安裝 ffmpeg。

以下是 Cloud Run Job 的運作方式插圖：



課程內容

- 如何使用 Dockerfile 建立容器映像檔，安裝第三方二進位檔
 - 如何為 Cloud Run 作業建立服務帳戶，藉此呼叫其他 Google Cloud 服務，並遵循最低權限原則
 - 如何從 Cloud Run Job 使用 Video Intelligence 用戶端程式庫
 - 如何呼叫 Google API，從 Vertex AI 取得每個場景的影像說明
-

步驟 2 · 約 0 分鐘

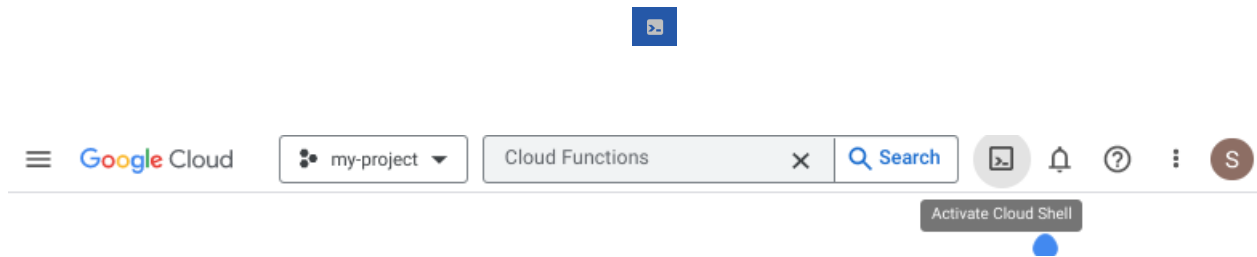
2. 設定和需求

必要條件

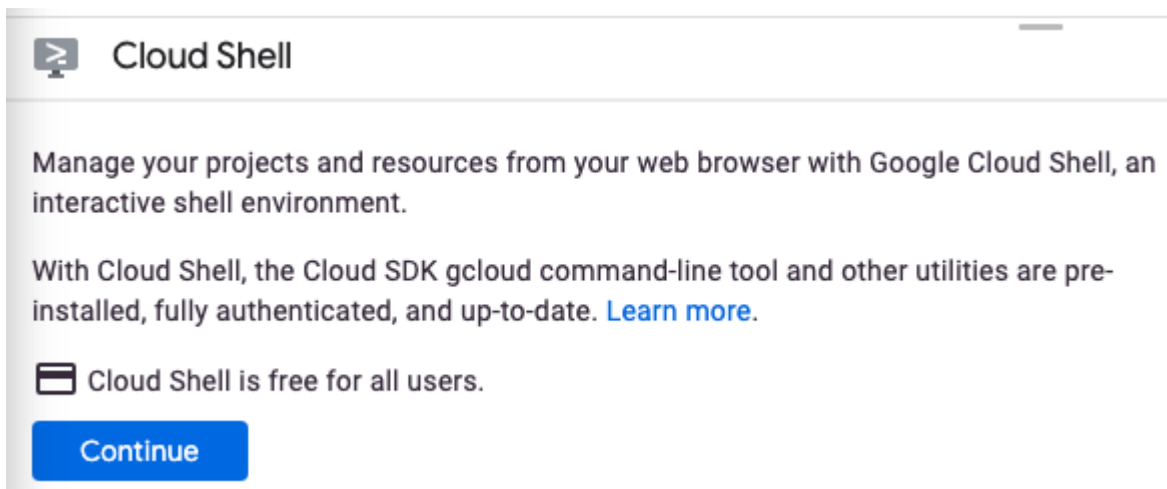
- 您已登入 Cloud Console。
- 您先前已部署 Cloud Run 服務。舉例來說，您可以按照[從原始碼部署網路服務的快速入門導覽課程](#)，開始使用 Cloud Run。

啟用 Cloud Shell

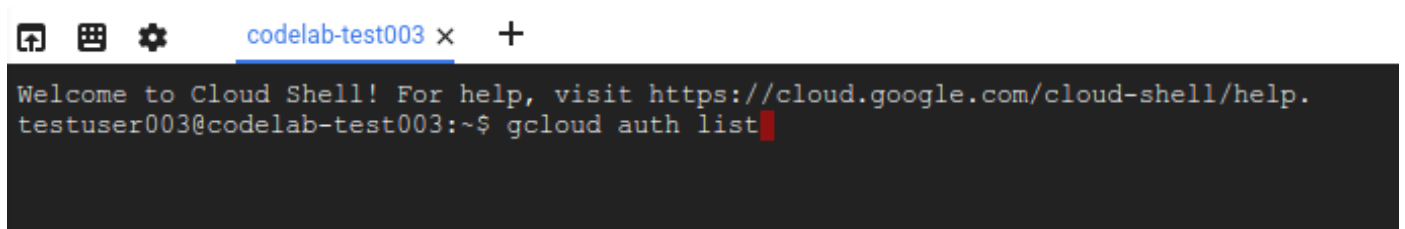
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

3. 啟用 API 並設定環境變數

開始進行本程式碼研究室之前，請先啟用數個 API。本程式碼研究室需要使用下列 API。執行下列指令即可啟用這些 API：

```
gcloud services enable run.googleapis.com \
  storage.googleapis.com \
  cloudbuild.googleapis.com \
  videointelligence.googleapis.com \
  aiplatform.googleapis.com
```

接著，您可以設定本程式碼研究室全程都會用到的環境變數。

```
REGION=<YOUR-REGION>
PROJECT_ID=<YOUR-PROJECT-ID>
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --
  format='value(projectNumber)')
JOB_NAME=video-describer-job
BUCKET_ID=$PROJECT_ID-video-describer
SERVICE_ACCOUNT="cloud-run-job-video"
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com
```

步驟 4 · 約 0 分鐘

4. 建立服務帳戶

您將為 Cloud Run 作業建立服務帳戶，以便存取 Cloud Storage、Vertex AI 和 Video Intelligence API。

首先，請建立服務帳戶。

```
gcloud iam service-accounts create $SERVICE_ACCOUNT \  
  --display-name="Cloud Run Video Scene Image Describer service account"
```

然後授予服務帳戶 Cloud Storage bucket 和 Vertex AI API 的存取權。

```
# to view & download storage bucket objects  
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
  --role=roles/storage.objectViewer  
  
# to call the Vertex AI imagetext model  
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
  --role=roles/aiplatform.user
```

步驟 5 · 約 0 分鐘

5. 建立 Cloud Storage 值區

使用下列指令建立 Cloud Storage bucket，以便上傳影片，供 Cloud Run 工作處理：

```
gsutil mb -l us-central1 gs://$BUCKET_ID/
```

[選用] 您可以下載這個[影片樣本](#)。

```
gsutil cp gs://cloud-samples-data/video/visionapi.mp4 testvideo.mp4
```

現在請將影片檔案上傳至儲存空間 bucket。

```
FILENAME=<YOUR-VIDEO-FILENAME>  
gsutil cp $FILENAME gs://$BUCKET_ID
```

步驟 6 · 約 0 分鐘

6. 建立 Cloud Run 工作

首先，請建立原始碼的目錄，然後 cd 到該目錄。

```
mkdir video-describer-job && cd $_
```

接著，請建立 `package.json` 檔案，並加入以下內容：

```
{
  "name": "video-describer-job",
  "version": "1.0.0",
  "private": true,
  "description": "describes the image in every scene for a given video",
  "main": "app.js",
  "author": "Google LLC",
  "license": "Apache-2.0",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "@google-cloud/storage": "^7.7.0",
    "@google-cloud/video-intelligence": "^5.0.1",
    "axios": "^1.6.2",
    "fluent-ffmpeg": "^2.1.2",
    "google-auth-library": "^9.4.1"
  }
}
```

這個應用程式包含多個來源檔案，可提高可讀性。首先，請使用以下內容建立 `app.js` 來源檔案。這個檔案包含作業的進入點，以及應用程式的主要邏輯。

注意：請將下列前兩行更新為您的 bucket ID 和影片檔案名稱。

```

const bucketName = "<YOUR_BUCKET_ID>";
const videoFilename = "<YOUR-VIDEO-FILENAME>";

const { captureImages } = require("./helpers/imageCapture.js");
const { detectSceneChanges } = require("./helpers/sceneDetector.js");
const { getImageCaption } = require("./helpers/imageCaptioning.js");
const storageHelper = require("./helpers/storage.js");
const authHelper = require("./helpers/auth.js");

const fs = require("fs").promises;
const path = require("path");

const main = async () => {

  try {

    // download the file to locally to the Cloud Run Job instance
    let localFilename = await storageHelper.downloadVideoFile(
      bucketName,
      videoFilename
    );

    // PART 1 - Use Video Intelligence API
    // detect all the scenes in the video & save timestamps to an array

    // EXAMPLE OUTPUT
    // Detected scene changes at the following timestamps:
    // [1, 7, 11, 12]
    let timestamps = await detectSceneChanges(localFilename);
    console.log(
      "Detected scene changes at the following timestamps: ",
      timestamps
    );

    // PART 2 - Use ffmpeg via dockerfile install
    // create an image of each scene change
    // and save to a local directory called "output"
    // returns the base filename for the generated images

    // EXAMPLE OUTPUT
    // creating screenshot for scene: 1 at output/video-filename-1.png
    // creating screenshot for scene: 7 at output/video-filename-7.png
    // creating screenshot for scene: 11 at output/video-filename-11.png
    // creating screenshot for scene: 12 at output/video-filename-12.png
    // returns the base filename for the generated images
    let imageBaseName = await captureImages(localFilename, timestamps);

    // PART 3a - get Access Token to call Vertex AI APIs via REST

```

```

// needed for the image captioning
// since we're calling the Vertex AI APIs directly
let accessToken = await authHelper.getAccessToken();
console.log("got an access token");

// PART 3b - use Image Captioning to describe each scene per screenshot
// EXAMPLE OUTPUT
/*
[
  {
    timestamp: 1,
    description:
      "an aerial view of a city with a bridge in the background"
  },
  {
    timestamp: 7,
    description:
      "a man in a blue shirt sits in front of shelves of donuts"
  },
  {
    timestamp: 11,
    description:
      "a black and white photo of people working in a bakery"
  },
  {
    timestamp: 12,
    description:
      "a black and white photo of a man and woman working in a bakery"
  }
]; */

// instantiate the data structure for storing the scene description and
timestamp
// e.g. an array of json objects,
// [{ timestamp: 5, description: "..."}], ...]
let scenes = [];

// for each timestamp, send the image to Vertex AI
console.log("getting Vertex AI description for each timestamps");
scenes = await Promise.all(
  timestamps.map(async (timestamp) => {
    let filepath = path.join(
      "./output",
      imageName + "-" + timestamp + ".png"
    );

    // get the base64 encoded image bc sending via REST
    const encodedFile = await fs.readFile(filepath, "base64");

```

```

        // send each screenshot to Vertex AI for description
        let description = await getImageCaption(
            accessToken,
            encodedFile
        );

        return { timestamp: timestamp, description: description };
    })
};

console.log("finished collecting all the scenes");
console.log(scenes);
} catch (error) {
    //return an error
    console.error("received error: ", error);
}
};

// Start script
main().catch((err) => {
    console.error(err);
});

```

接著，請建立 `Dockerfile` 。

```

# Copyright 2020 Google, LLC.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

# Use the official lightweight Node.js image.
# https://hub.docker.com/_/node
FROM node:20.10.0-slim

# Create and change to the app directory.
WORKDIR /usr/src/app

RUN apt-get update && apt-get install -y ffmpeg

# Copy application dependency manifests to the container image.
# A wildcard is used to ensure both package.json AND package-lock.json are copied.
# Copying this separately prevents re-running npm install on every code change.
COPY package*.json ./

# Install dependencies.
# If you add a package-lock.json speed your build by switching to 'npm ci'.
# RUN npm ci --only=production
RUN npm install --production

# Copy local code to the container image.
COPY . .

# Run the job on container startup.
CMD [ "npm", "start" ]

```

並建立名為 `.dockerignore` 的檔案，忽略特定檔案的容器化作業。

```

Dockerfile
.dockerignore
node_modules
npm-debug.log

```

現在請建立名為 `helpers` 的資料夾。這個資料夾會包含 5 個輔助檔案。

```
mkdir helpers  
cd helpers
```

接著，請建立 `sceneDetector.js` 檔案，並加入以下內容。這個檔案會使用 Video Intelligence API，偵測影片中的場景變化。

```

const fs = require("fs");
const util = require("util");
const readFile = util.promisify(fs.readFile);
const ffmpeg = require("fluent-ffmpeg");

const Video = require("@google-cloud/video-intelligence");
const client = new Video.VideoIntelligenceServiceClient();

module.exports = {
  detectSceneChanges: async function (downloadedFile) {
    // Reads a local video file and converts it to base64
    const file = await readFile(downloadFile);
    const inputContent = file.toString("base64");

    // setup request for shot change detection
    const videoContext = {
      speechTranscriptionConfig: {
        languageCode: "en-US",
        enableAutomaticPunctuation: true
      }
    };

    const request = {
      inputContent: inputContent,
      features: ["SHOT_CHANGE_DETECTION"]
    };

    // Detects camera shot changes
    const [operation] = await client.annotateVideo(request);
    console.log("Shot (scene) detection in progress...");
    const [operationResult] = await operation.promise();

    // Gets shot changes
    const shotChanges =
      operationResult.annotationResults[0].shotAnnotations;

    console.log(
      "Shot (scene) changes detected: " + shotChanges.length
    );

    // data structure to be returned
    let sceneChanges = [];

    // for the initial scene
    sceneChanges.push(1);

    // if only one scene, keep at 1 second
    if (shotChanges.length === 1) {

```

```

        return sceneChanges;
    }

    // get length of video
    const videoLength = await getVideoLength(downloadedFile);

    shotChanges.forEach((shot, shotIndex) => {
        if (shot.endTimeOffset === undefined) {
            shot.endTimeOffset = {};
        }
        if (shot.endTimeOffset.seconds === undefined) {
            shot.endTimeOffset.seconds = 0;
        }
        if (shot.endTimeOffset.nanos === undefined) {
            shot.endTimeOffset.nanos = 0;
        }

        // convert to a number
        let currentTimestampSecond = Number(
            shot.endTimeOffset.seconds
        );

        let sceneChangeTime = 0;
        // double-check no scenes were detected within the last second
        if (currentTimestampSecond + 1 > videoLength) {
            sceneChangeTime = currentTimestampSecond;
        } else {
            // otherwise, for simplicity, just round up to the next second
            sceneChangeTime = currentTimestampSecond + 1;
        }

        sceneChanges.push(sceneChangeTime);
    });

    return sceneChanges;
}

};

async function getVideoLength(localFile) {
    let getLength = util.promisify(ffmpeg.ffprobe);
    let length = await getLength(localFile);

    console.log("video length: ", length.format.duration);
    return length.format.duration;
}

```

現在請建立名為 `imageCapture.js` 的檔案，並加入下列內容。這個檔案會使用節點套件 [fluent-ffmpeg](#)，從節點應用程式內執行 `ffmpeg` 指令。


```

const ffmpeg = require("fluent-ffmpeg");
const path = require("path");
const util = require("util");

module.exports = {
  captureImages: async function (localFile, scenes) {
    let imageBaseName = path.parse(localFile).name;

    try {
      for (scene of scenes) {
        console.log("creating screenshot for scene: ", +scene);
        await createScreenshot(localFile, imageBaseName, scene);
      }
    } catch (error) {
      console.log("error gathering screenshots: ", error);
    }

    console.log("finished gathering the screenshots");
    return imageBaseName; // return the base filename for each image
  }
};

async function createScreenshot(localFile, imageBaseName, scene) {
  return new Promise((resolve, reject) => {
    ffmpeg(localFile)
      .screenshots({
        timestamps: [scene],
        filename: `${imageBaseName}-${scene}.png`,
        folder: "output",
        size: "320x240"
      })
      .on("error", () => {
        console.log(
          "Failed to create scene for timestamp: " + scene
        );
        return reject(
          "Failed to create scene for timestamp: " + scene
        );
      })
      .on("end", () => {
        return resolve();
      });
  });
}

```

最後，建立名為 `imageCaptioning.js` 的檔案，並加入下列內容。這個檔案會使用 Vertex AI 取得每個場景圖片的視覺說明。

```

const axios = require("axios");
const { GoogleAuth } = require("google-auth-library");

const auth = new GoogleAuth({
  scopes: "https://www.googleapis.com/auth/cloud-platform"
});

module.exports = {
  getImageCaption: async function (token, encodedFile) {
    // this example shows you how to call the Vertex REST APIs directly
    // https://cloud.google.com/vertex-ai/generative-ai/docs/image/image-
captioning#get-captions-short
    // https://cloud.google.com/vertex-ai/generative-ai/docs/model-
reference/image-captioning

    let projectId = await auth.getProjectId();

    let config = {
      headers: {
        "Authorization": "Bearer " + token,
        "Content-Type": "application/json; charset=utf-8"
      }
    };

    const json = {
      "instances": [
        {
          "image": {
            "bytesBase64Encoded": encodedFile
          }
        }
      ],
      "parameters": {
        "sampleCount": 1,
        "language": "en"
      }
    };

    let response = await axios.post(
      "https://us-central1-aiplatform.googleapis.com/v1/projects/" +
        projectId +
        "/locations/us-central1/publishers/google/models/imagetext:predict",
      json,
      config
    );

    return response.data.predictions[0];
  }
};

```

```
}  
};
```

建立名為 `auth.js` 的檔案。這個檔案會使用 Google 驗證用戶端程式庫，取得直接呼叫 Vertex AI 端點所需的存取權杖。

```
const { GoogleAuth } = require("google-auth-library");  
  
const auth = new GoogleAuth({  
  scopes: "https://www.googleapis.com/auth/cloud-platform"  
});  
  
module.exports = {  
  getAccessToken: async function () {  
    return await auth.getAccessToken();  
  }  
};
```

最後，建立名為 `storage.js` 的檔案。這個檔案會使用 Cloud Storage 用戶端程式庫，從 Cloud Storage 下載影片。

```
const { Storage } = require("@google-cloud/storage");  
  
module.exports = {  
  downloadVideoFile: async function (bucketName, videoFilename) {  
    // Creates a client  
    const storage = new Storage();  
  
    // keep same name locally  
    let localFilename = videoFilename;  
  
    const options = {  
      destination: localFilename  
    };  
  
    // Download the file  
    await storage  
      .bucket(bucketName)  
      .file(videoFilename)  
      .download(options);  
  
    console.log(  
      `gs://${bucketName}/${videoFilename} downloaded locally to  
      ${localFilename}.`  
    );  
  
    return localFilename;  
  }  
};
```


步驟 7 · 約 0 分鐘

7. 部署及執行 Cloud Run 工作

首先，請確認您位於程式碼研究室的根目錄 `video-describer-job`。

```
cd .. && pwd
```

接著，您可以使用這個指令部署 Cloud Run 作業。

```
gcloud run jobs deploy $JOB_NAME --source . --region $REGION
```

現在，您可以執行下列指令來執行 Cloud Run Job：

```
gcloud run jobs execute $JOB_NAME
```

工作執行完畢後，您可以執行下列指令，取得記錄 URI 的連結。(或者，您也可以使用 Cloud 控制台，直接前往 Cloud Run Jobs 查看記錄。)

```
gcloud run jobs executions describe <JOB_EXECUTION_ID>
```

記錄中應會顯示下列輸出內容：

```
[{ timestamp: 1, description: 'what is google cloud vision api ? is written on a  
white background .' },  
{ timestamp: 3, description: 'a woman wearing a google cloud vision api shirt sits at  
a table' },  
{ timestamp: 18, description: 'a person holding a cell phone with the words what is  
cloud vision api on the bottom' }, ...]
```

步驟 8 · 約 0 分鐘

8. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Video Intelligence API](#)、[Cloud Run](#) 和 [Vertex AI 圖像說明生成](#) 的說明文件。

涵蓋內容

- 如何使用 Dockerfile 建立容器映像檔，安裝第三方二進位檔
 - 如何為 Cloud Run 作業建立服務帳戶，藉此呼叫其他 Google Cloud 服務，並遵循最低權限原則
 - 如何從 Cloud Run Job 使用 Video Intelligence 用戶端程式庫
 - 如何呼叫 Google API，從 Vertex AI 取得每個場景的影像說明
-

9. 清除所用資源

為避免產生意外費用 (例如，如果這個 Cloud Run 工作意外遭到呼叫的次數，超過[免費層級的每月 Cloud Run 呼叫配額](#))，您可以刪除 Cloud Run 工作，或刪除您在步驟 2 中建立的專案。

如要刪除 Cloud Run 工作，請前往 Cloud Run Cloud 控制台 (<https://console.cloud.google.com/run/>)，然後刪除 `video-describer-job` 函式 (如果您使用其他名稱，請刪除 `$JOB_NAME`)。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
